CO5_ASSESSMENT 1

CONSTRAINT -BASED PROBLEM SOLVING

Name : Adhi Kesavan S **Course :** Computer Vision

Reg No : 192521172 **Course Code** : ITA0503

1. Real-Time Face Recognition for Campus Security

A university wants to deploy a camera-based security system at the entrance that can identify registered students and staff in real time.
The system should:

- Capture faces from live CCTV/video streams.
- Detect faces under different lighting conditions.
- Extract suitable facial features.
- Recognize registered individuals.
- Maintain real-time performance on a low-cost computer.
- Minimize false recognition.

Design a complete OpenCV-based face recognition pipeline from image acquisition to final identification. Explain the preprocessing, face detection, feature extraction, recognition, and optimization techniques used. Justify how your system achieves accuracy and real-time performance.

2. Intelligent Traffic Violation Detection System

A traffic department wants to automatically monitor vehicles using roadside cameras.
The system should:

- Detect vehicles from live video.
- Track vehicles across consecutive frames.
- Identify vehicles crossing a predefined line.
- Count vehicles accurately.
- Detect abnormal or unwanted movements.
- Display vehicle count and alerts in real time.

Design an OpenCV-based solution incorporating video acquisition, preprocessing, object detection, motion analysis, tracking, and visualization. Explain how you would handle shadows, camera noise, dynamic backgrounds, and multiple vehicles to minimize counting errors.

3. Automated Document Processing System

A company receives thousands of scanned documents that may contain blurred text, uneven illumination, noise, and different font sizes.
The system should:

- Automatically improve document quality.
- Remove noise and unwanted background.
- Convert the document into a suitable format for OCR.

- Segment text regions.
- Extract readable text efficiently.
- Process a large number of documents in batch mode.

Design an OpenCV-based image processing pipeline for this system. Explain the role of grayscale conversion, thresholding, filtering, morphological operations, segmentation, and geometric correction. Justify how each stage improves OCR accuracy.

4. Real-Time Multi-Person Tracking and Attendance System

A college wants to automatically record student attendance using classroom CCTV cameras.
The system should:

- Detect multiple faces simultaneously.
- Track each person across video frames.
- Recognize registered students.
- Avoid repeatedly marking the same student.
- Handle slight head movement and temporary face disappearance.
- Operate continuously with minimum processing delay.

Design a complete OpenCV-based system using face detection, feature extraction, recognition, and tracking. Explain how you maintain identity across frames and prevent false or duplicate attendance records.

5. Smart Home Surveillance and Intrusion Detection

A home security system uses a fixed camera to monitor an entrance continuously.
The system should:

- Detect significant motion.
- Distinguish actual movement from camera noise and small background changes.
- Track moving objects.
- Identify suspicious activity.
- Generate an alert when an unauthorized person enters a predefined region.
- Operate in real time with limited computational resources.

Design an OpenCV-based surveillance pipeline using background subtraction, filtering, morphological operations, contour/feature detection, motion analysis, and tracking. Explain how your approach reduces false alarms caused by shadows, illumination changes, trees, and other dynamic background objects.

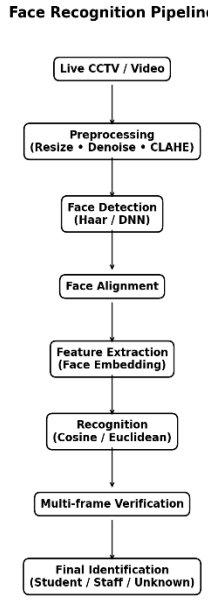
Solution :

1. Real-Time Face Recognition for Campus Security

Introduction

A real-time face recognition system can be used at a university entrance to identify registered students and staff from CCTV cameras. The system continuously captures video, detects faces, extracts important facial features, compares them with registered users, and displays the person's identity.

Suitable System Pipeline



Share

Download: Face Recognition Pipeline Diagram

Proposed OpenCV Pipeline

CCTV Camera → Preprocessing → Face Detection → Face Alignment → Feature Extraction → Face Recognition → Verification → Identification

1. Image Acquisition

The CCTV camera continuously captures video frames. OpenCV's VideoCapture() can be used to obtain frames from a webcam, CCTV stream, or video file.

The frame size can be reduced to approximately 640 × 480 to decrease processing time.

2. Preprocessing

The captured image may contain noise and uneven lighting. Therefore, preprocessing is performed before detecting the face.

The following operations can be used:

- Resize the image.
- Convert the image to grayscale when appropriate.
- Remove noise using Gaussian or median filtering.
- Improve contrast using histogram equalization or CLAHE.
- Normalize brightness.

For example:

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

gray = cv2.equalizeHist(gray)

This makes faces easier to detect under different lighting conditions.

3. Face Detection

The next step is to locate faces in the image.

OpenCV can use:

- Haar Cascade Classifier
- OpenCV DNN face detector

For a low-cost computer, a lightweight detector is preferred because it requires less processing power.

The detector returns the bounding box of each face:

(x, y, width, height)

4. Face Alignment

A person's face may be slightly tilted or rotated. Facial landmarks such as the eyes, nose and mouth can be used to align the face.

The aligned face is then resized to a standard size.

This improves recognition because the stored and live faces have similar orientations.

5. Feature Extraction

The complete face image is not directly compared. Instead, important facial characteristics are converted into a numerical feature vector called a face embedding.

For example:

Student A → [0.12, 0.45, 0.78, ...]

Student B → [0.31, 0.62, 0.19, ...]

A suitable face-embedding model can be used with OpenCV.

6. Face Recognition

The feature vector obtained from the live camera is compared with the registered feature vectors.

Possible comparison methods include:

- Euclidean distance
- Cosine similarity
- K-Nearest Neighbors
- SVM

If the similarity is above the required threshold, the person is identified.

Otherwise, the person is classified as Unknown.

7. Minimizing False Recognition

False recognition is an important problem in security systems. The following methods can reduce it:

- Use a strict recognition threshold.
- Reject very small or poor-quality faces.
- Compare against multiple reference images.
- Confirm the identity over several consecutive frames.
- Use an "Unknown" class rather than forcing every face to match.
- Avoid recognition when the face is partially hidden.

For example:

Frame 1 → Student A

Frame 2 → Student A

Frame 3 → Student A

↓

Confirmed Student A

8. Real-Time Optimization

Since the system runs on a low-cost computer, optimization is important.

Techniques include:

1. Resize the video frames.
2. Detect faces every few frames.
3. Track faces between detections.
4. Use lightweight recognition models.
5. Process only the entrance region.
6. Avoid unnecessary image operations.
7. Use GPU acceleration if available.

Final Output

The system can display:

CAMPUS SECURITY SYSTEM

Name : Adhikesavan

Category : Student

ID : 23IT104

Status : Recognized

Confidence : 96%

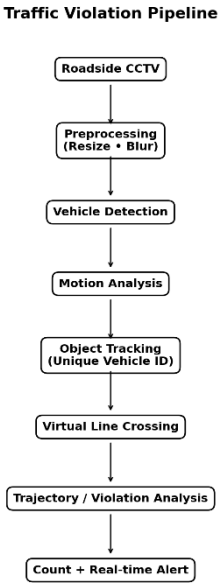
The proposed system combines image preprocessing, face detection, alignment, feature extraction, recognition and tracking. Lighting correction improves detection quality, while multi-frame verification and suitable thresholds reduce false recognition. Frame resizing and tracking reduce computational requirements, allowing the system to operate in real time on a low-cost computer.

2. Intelligent Traffic Violation Detection System

Introduction

An intelligent traffic monitoring system can automatically detect and track vehicles using roadside CCTV cameras. It can count vehicles, determine their direction and detect violations such as wrong-way movement or crossing restricted areas.

Suitable System Pipeline



Share

Download: Traffic Violation Detection Pipeline

Proposed OpenCV Pipeline

CCTV → Preprocessing → Vehicle Detection → Motion Analysis → Tracking → Line Crossing → Violation Detection → Counting and Alerts

1. Video Acquisition

OpenCV captures frames from a roadside camera.

```
cap = cv2.VideoCapture("traffic.mp4")
```

The same approach can be used with a live CCTV/IP stream.

2. Preprocessing

Traffic video may contain camera noise and unnecessary details.

The following operations can be performed:

- Resize the frame.
- Convert to grayscale when appropriate.
- Apply Gaussian blur.
- Improve image quality.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

3. Vehicle Detection

Vehicles can be detected using:

- Background subtraction.
- Contour detection.
- OpenCV DNN object detection.

The detector identifies cars, buses, trucks and motorcycles.

4. Motion Analysis

For a fixed roadside camera, background subtraction can identify moving vehicles.

OpenCV provides:

- MOG2
- KNN

Example:

fgmask = backSub.apply(frame)

The foreground mask contains regions that are different from the background.

5. Morphological Processing

The foreground mask can contain small noise and broken regions.

Morphological operations help clean it.

- Opening removes small noise.
- Closing fills small gaps.
- Dilation connects vehicle regions.
- Erosion removes unwanted pixels.

6. Vehicle Tracking

Each vehicle is assigned a unique tracking ID.

For example:

Car → ID 01

Bus → ID 02

Car → ID 03

Truck → ID 04

Tracking can be performed using:

- Centroid tracking.
- Kalman filtering.
- Optical flow.
- SORT-type tracking algorithms.

This is important because a vehicle should not be counted again in every frame.

7. Line-Crossing Detection

A virtual line is drawn across the road.

```
-------------------------------
      Virtual Line
-------------------------------
          ↓
        Vehicle
```

The position of each vehicle's centroid is monitored.

When the centroid moves from one side of the line to the other, the system considers that vehicle to have crossed the line.

The vehicle's unique ID is stored so that it is counted only once.

8. Detecting Abnormal Movement

The system can analyze the trajectory of each vehicle.

It can identify:

- Wrong-way driving.
- Illegal lane changes.
- Sudden stops.
- U-turns.
- Entry into restricted areas.
- Abnormal trajectories.

9. Handling Shadows

Shadows can be incorrectly detected as vehicles.

To reduce this problem:

- Use shadow detection in background subtraction.
- Analyze object size and shape.
- Apply morphological operations.
- Ignore very small regions.
- Use object detection together with motion detection.

10. Handling Camera Noise

Camera noise can be reduced using:

- Gaussian filtering.
- Median filtering.
- Morphological opening.

This prevents small noisy regions from being detected as vehicles.

11. Handling Dynamic Backgrounds

Trees, moving advertisements and changing lighting can cause background changes.

An adaptive background model such as MOG2 or KNN can be used. Small and short-lived changes should also be ignored.

12. Visualization

The output can display:

TRAFFIC MONITORING SYSTEM

Vehicle Count : 48

Vehicle ID 21 : Normal

Vehicle ID 22 : Normal

Vehicle ID 23 : WRONG DIRECTION

ALERT!

The system combines video acquisition, preprocessing, vehicle detection, motion analysis and object tracking. Unique tracking IDs prevent duplicate counting, while virtual line detection identifies vehicles crossing a particular location. Filtering, adaptive background subtraction and object-size checks reduce errors caused by shadows, noise and dynamic backgrounds.

3. Automated Document Processing System

Introduction

Companies often receive thousands of scanned documents containing noise, blurred text, uneven illumination and different font sizes. OpenCV can preprocess these documents before sending them to an OCR system.

Suitable System Pipeline

Download: Document Processing Pipeline

Proposed OpenCV Pipeline

Document → Grayscale → Noise Removal → Illumination Correction → Thresholding → Morphology → Segmentation → Geometric Correction → OCR

1. Image Acquisition

Documents may be obtained from:

- Scanners.
- Mobile cameras.
- Digital documents.
- PDF-to-image conversion.

Each document is converted into an image before processing.

2. Grayscale Conversion

Most document processing does not require color information.

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

Grayscale conversion:

- Reduces data size.
- Simplifies processing.
- Makes thresholding easier.
- Improves processing speed.

3. Noise Removal

Scanned documents may contain:

- Salt-and-pepper noise.
- Scanner noise.
- Small dots.
- Compression artifacts.

Median filtering is particularly useful for removing salt-and-pepper noise.

filtered = cv2.medianBlur(gray, 3)

4. Illumination Correction

A photograph of a document may contain dark corners or shadows.

CLAHE or background normalization can improve local contrast.

This makes text more visible even when the document has uneven lighting.

5. Thresholding

Thresholding converts the grayscale document into a binary image.

For uneven lighting, adaptive thresholding is useful.

```
binary = cv2.adaptiveThreshold(
    filtered,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)
```

The result contains:

Text → Black

Background → White

6. Morphological Operations

Morphological operations improve the structure of text.

Opening:
Removes small unwanted noise.

Closing:
Connects broken characters.

Dilation:
Makes thin text stronger.

Erosion:
Removes small unwanted regions.

7. Text Segmentation

The document is divided into meaningful regions.

For example:

Document

↓

Paragraphs

↓

Text Lines

↓

Words

↓

Characters

Contours, connected components or projection profiles can be used for segmentation.

8. Geometric Correction

A document photographed at an angle may have perspective distortion.

The system can:

1. Detect the document boundary.
2. Find its corner points.
3. Apply perspective transformation.
4. Correct the document orientation.
5. Deskew the text.

This makes the document rectangular and easier for OCR to understand.

9. OCR

After preprocessing, the clean document image is given to an OCR engine.

The OCR system converts the image into readable text.

Processed Image

↓

OCR

↓

"University Examination

Registration Form"

10. Batch Processing

Thousands of documents can be processed automatically:

Input Folder

↓

Document 1 → Process → OCR → Text

Document 2 → Process → OCR → Text

Document 3 → Process → OCR → Text

.

.

Document N → Process → OCR → Text

Batch processing reduces manual work.

Role of Each Processing Stage

Stage	Purpose
Grayscale	Simplifies the image
Filtering	Removes noise
Illumination correction	Improves uneven lighting
Thresholding	Separates text and background
Morphology	Repairs/cleans text
Segmentation	Separates text regions
Deskewing	Corrects tilted documents
Perspective correction	Corrects camera distortion
OCR	Extracts readable text

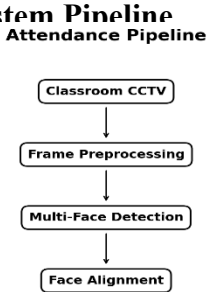
The proposed document-processing pipeline improves OCR accuracy by cleaning the document before recognition. Grayscale conversion simplifies the image, filtering removes noise, thresholding separates text from the background, morphology repairs text regions, and geometric correction removes distortion. Batch processing makes the system suitable for processing thousands of documents efficiently.

4. Real-Time Multi-Person Tracking and Attendance System

Introduction

A classroom attendance system can use CCTV cameras to identify students automatically. The major challenge is to maintain each student's identity while they move, turn their heads or temporarily disappear behind another person.

Suitable System Pipeline



Download: Multi-Person Attendance Pipeline

Proposed OpenCV Pipeline

CCTV → Preprocessing → Multi-Face Detection → Alignment → Feature Extraction → Recognition → Tracking → Verification → Attendance Database

1. Video Acquisition

A classroom CCTV camera continuously captures video frames.

cap = cv2.VideoCapture(0)

The system processes the frames continuously.

2. Preprocessing

The frames can be:

- Resized.
- Denoised.
- Contrast enhanced.
- Converted to grayscale when appropriate.

This improves detection quality and reduces processing requirements.

3. Multi-Face Detection

The system must detect multiple students in the same frame.

For example:

Classroom Frame

├── Student A

├── Student B

├── Student C

├── Student D

└── Student E

OpenCV Haar Cascade or DNN-based face detection can be used.

4. Face Alignment

Each detected face is aligned based on facial landmarks.

This helps when students have slight head rotations.

5. Feature Extraction

Each face is converted into a numerical feature vector.

Student A → Feature Vector A

Student B → Feature Vector B

Student C → Feature Vector C

These vectors are compared against registered student templates.

6. Recognition

Recognition can use:

- Cosine similarity.
- Euclidean distance.
- KNN.
- SVM.

The system assigns a student ID only when the similarity is sufficiently high.

7. Tracking

After recognizing a student, the system assigns a tracking ID.

Student A → Track ID 01

Student B → Track ID 02

Student C → Track ID 03

The system then tracks their movement between frames.

This reduces the need to perform expensive face recognition on every frame.

8. Temporary Face Disappearance

Sometimes a student may:

- Turn their head.
- Bend down.

- Be temporarily blocked.
- Move outside the camera's view.

The system should not immediately delete the tracking identity.

Instead:

Face Disappears

↓

Keep Track ID

↓

Wait for Several Frames

↓

Face Appears Again

↓

Continue Existing Track

This prevents duplicate identities.

9. Preventing Duplicate Attendance

Attendance should be stored using the student's unique ID.

For example:

```
if student_id not in attendance:
    attendance.add(student_id)
```

Once a student is marked present, later detections do not create another attendance record.

10. False Recognition Prevention

The following methods improve reliability:

- Confidence threshold.
- Multi-frame recognition.
- Minimum face size.
- Face-quality checking.
- Unknown-person classification.
- Temporal consistency.
- Multiple reference images per student.

For example:

Frame 1 → Student A

Frame 2 → Student A

Frame 3 → Student A

↓

Confirmed Student

↓

Mark Attendance

11. Real-Time Optimization

To reduce processing delay:

- Resize frames.
- Detect faces periodically.
- Track faces between detection frames.
- Use lightweight models.
- Process only the classroom region.
- Use GPU acceleration where available.

Example Output

==========

CLASSROOM ATTENDANCE

==========

Student A PRESENT

Student B PRESENT

Student C PRESENT

Student D ABSENT

Present: 3

Absent : 1

==========

The proposed system combines face detection, feature extraction, recognition and tracking. Tracking maintains student identities across consecutive frames, while multi-frame confirmation reduces false recognition. A unique student ID and attendance database ensure that each student

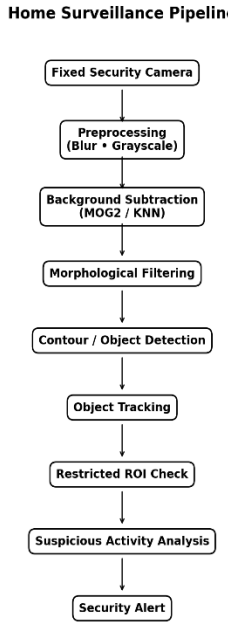
is marked only once. Frame skipping and tracking also help the system operate continuously with low processing delay.

5. Smart Home Surveillance and Intrusion Detection

Introduction

A smart home surveillance system can use a fixed camera to monitor an entrance continuously. Instead of treating every small change as an intrusion, the system analyzes motion, object size, location and movement over time.

Suitable System Pipeline



Download: Smart Home Surveillance Pipeline

Proposed OpenCV Pipeline

Camera → Preprocessing → Background Subtraction → Morphological Filtering → Contour Detection → Object Tracking → ROI Analysis → Suspicious Activity Detection → Alert

1. Camera Acquisition

A fixed CCTV camera continuously captures the entrance.

cap = cv2.VideoCapture(0)

A fixed camera is particularly suitable for background subtraction because the background remains mostly unchanged.

2. Preprocessing

Each frame can be processed using:

- Grayscale conversion.

- Gaussian filtering.
- Noise removal.
- Image resizing.

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

3. Background Subtraction

A background model represents the normal scene.

OpenCV provides:

- MOG2.
- KNN.

Example:

```
fgmask = backSub.apply(frame)
```

Moving objects appear in the foreground mask.

4. Morphological Processing

The foreground mask can contain small unwanted regions.

Morphological operations are used to clean it.

Foreground Mask

↓

Opening

↓

Closing

↓

Clean Motion Regions

Opening removes small noise, while closing fills small gaps in moving objects.

5. Contour Detection

Contours are extracted from the cleaned foreground mask.

```
contours, _ = cv2.findContours(
   mask,
   cv2.RETR_EXTERNAL,
   cv2.CHAIN_APPROX_SIMPLE
```

)

Very small contours are ignored.

Larger contours are treated as possible moving objects.

6. Object Tracking

Every significant moving object receives a tracking ID.

Person → ID 01

Dog → ID 02

Vehicle → ID 03

Possible tracking methods include:

- Centroid tracking.
- Kalman filter.
- Optical flow.
- CSRT/KCF trackers.

7. Restricted Region of Interest

A region around the entrance is defined as the protected area.

The system checks whether a tracked object enters this region.

8. Suspicious Activity Detection

The system can identify:

- Unauthorized entry.
- Movement during restricted hours.
- Loitering.
- Repeated movement around the entrance.
- Movement in an unexpected direction.

The decision can be based on multiple conditions rather than a single moving pixel.

9. Reducing False Alarms

Camera Noise

Use Gaussian or median filtering and ignore very small contours.

Shadows

Use shadow detection and analyze the shape and intensity of moving regions.

Trees and Leaves

Small and irregular moving regions can be ignored using contour-area thresholds.

Illumination Changes

Adaptive background subtraction can update the background model gradually.

Temporary Changes

Require motion to remain present for several consecutive frames before generating an alert.

10. Alert Generation

When a suspicious object enters the protected region:

```
==================================

      SECURITY ALERT

==================================

Unauthorized movement detected!


Object ID : 03

Location  : Entrance ROI

Status    : Suspicious

==================================
```

The system can also save the relevant frame or a short video clip.

11. Real-Time Optimization

For a low-cost computer:

- Reduce frame resolution.
- Process selected frames.
- Restrict processing to the entrance ROI.
- Use lightweight tracking.
- Avoid unnecessary deep-learning operations.
- Detect objects periodically and track them between detections.

The proposed smart surveillance system uses background subtraction, filtering, morphology, contour detection and tracking to identify significant movement. Shadows, trees, camera noise and small background changes are reduced using filtering, area thresholds and temporal confirmation. ROI-based processing and lightweight algorithms make the system suitable for continuous real-time operation on limited hardware.